SLIIT
Discover Your Future

BSc (Hons) in Information Technology Specializing Data Science

Research Project - IT4010 MLOPs Report

Group ID: R26-DS-006

Project Title: AI-Based Adaptive VR Meditation System

1. **IT22231314**: Safety constrained adaptive VR environment
2. **IT22191038**: Cosinor-Enhanced Multi-level Stress Inference from Wearable RR Intervals under Real-Time Deployment Constraints
3. **IT22172396**: Confidence Aware Personalized Adaptive Audio Control
4. **IT22621788**: Multilingual Voice-Based Stress Assessment with Cross-Modal Session Validation

		IT22231314	IT22191038	IT22172396	IT22621788
Data Pipeline	**Data Sources:**	• This component does not use a public dataset, because the physiology datasets are handled by the HRV stress inference component. • It receives the HRV stress inference component's processed stress predictions over WebSocket (/stream) about once every 60 seconds. Each message contains a timestamp, heart rate, optional RMSSD/SDNN, four class probabilities, a continuous stress score and a signal-quality value. • The mobile app sends a pseudonymous participant and session ID, the study condition, five normalised visual preferences and session commands (start, pause, resume, stop and emergency stop)	• The model is trained on WESAD (15 subjects, about 1.5–2 hours each). RR intervals come from the chest ECG at 700 Hz and temperature from the Empatica E4 wrist device, giving 12,026 labelled 60-beat windows. • The Hongn et al. stress and exercise dataset from PhysioNet (Empatica E4 BVP at 64 Hz, 33	• The component receives the HRV stress inference component's accepted physiology windows about once every 60 seconds. These contain normalised stress, confidence, signal quality, heart rate and optional RMSSD/SDNN, but no raw PPG. • The mobile app sends a participant pseudonym, long-term audio preferences and optional session-only overrides through the session relay. • The Temple Pond scene provides the musical	• The shipped model is trained on 12,434 clips of natural conversational speech from MELD. • About 11,688 acted clips from RAVDESS, CREMA-D and TESS were used for the ablation study. • The model is tested on a real-voice set of 24 clips (17 stressed and 7 calm, including 11 Sinhala clips

		through the mindsync-session-v3 relay contract. • Unity itself supplies the current five-dimensional environment state, the scene's safe default, the action history and the safety state. • No participant data has been collected yet, so the tests use synthetic fixtures and a local physiology simulator. The planned pilot involves 5–7 adults, each completing two 20-minute visits.	participants) is used only for the cross-dataset replication, not for training. • During live use, the ESP32-S3 wearable with a MAX30100 sensor sends PPG at 100 Hz over BLE to the mobile app. The app resamples it to 64 Hz and forwards 15-second frames of 960 samples to /ingest.	reference and a fixed ambient MP3, and Google Lyria (lyria-3-clip-preview) generates the meditation music. • No public dataset is used, and no real-participant dataset exists yet. The archived simulator used synthetic users only.	used zero-shot), and 26 additional Sinhala clips were collected from 7 speakers. • During live use, the React Native app records the phone microphone as 16 kHz mono 16-bit PCM WAV, with automatic gain control and noise suppression turned off.
	Data Preprocessing:	• Raw PPG cleaning, HRV calculation and stress labelling are done upstream by the HRV stress inference component, so they are not repeated here. • Unity C# validates every incoming window and rejects malformed, duplicate, out-of-order, future-dated, stale (older than 90 s), short (under 30 s), probability-inconsistent (sum error above 0.005) or low-quality (below 0.50) windows. Up to 8	• R-peaks are detected in the chest ECG with NeuroKit2, and the resulting RR series goes through artefact rejection. • The RR series is cut into 60-beat windows with a 5-beat step, and each window	• Duplicate, out-of-order, stale, overlapping or low-quality windows are rejected. • Learning only starts after two baseline windows, and each window must be 30–120 seconds long with confidence of at least 0.45 and signal quality of at least 0.60.	• Audio is resampled to 16 kHz, loudness-normalised to an RMS of 0.05, high-pass filtered at 70 Hz and trimmed of silence. • A quality gate checks RMS level, clipping ratio, SNR and

		accepted windows are kept in a buffer. • The continuous stress score is rescaled from [0, 3] to [0, 1]. A 24-value feature vector (adaptive-vr-policy-features/1.0) is then built from the stress values, probabilities, signal quality, user preferences, current state and normalised distances. No augmentation or relabelling is applied.	takes the label of its last beat (endpoint labelling). Four ordinal stress levels are built from the protocol phase and local HRV. • Twenty-five features are computed, covering time-domain, frequency-domain and Poincaré measures, residuals against causal EWMA baselines (half-lives of 60, 300 and 1800 beats) and time-of-day terms. They are scaled with a StandardScaler, and a 7-channel sequence is prepared for the neural network. • In live use, NeuroKit2	• Preferences, applied audio controls, physiology availability and session context are encoded into a bounded 40-value vector (audio-context-v1-40). Missing HRV is flagged rather than imputed.	voice activity (Silero VAD). • Two feature sets are extracted: frozen emotion2vec_plus_large embeddings (through funasr) and 12 prosody features such as pitch, jitter, shimmer, HNR, speech rate and pauses (through parselmouth and librosa). The features are cached as .npz files.

			ppg_clean and ppg_peaks turn each PPG frame into RR intervals, which pass through exactly the same causal feature code. Parity tests confirm that the live features match the training features.		
	Data storage and versioning	• The Quest stores session telemetry as JSON Lines files (AdaptiveMeditationTelemetry/<sessionId>.jsonl). The relay also keeps a session JSONL record, which the mobile app can upload to Supabase. • The C# code, schemas, synthetic fixtures and ScriptableObject configurations are versioned in Git. • Each schema and profile has a version ID, such as temple-pond-pilot-freeze-v2, adaptive-vr-telemetry/1.1-pilot and adaptive-vr-policy-features/1.0. Telemetry records these IDs so every run can be traced to an exact Git revision. DVC and Git-LFS are not used.	• Raw datasets are not stored in Git because of the WESAD licence. They are kept as Kaggle input datasets and on a local machine. • Code, notebooks and model artifacts are versioned on GitHub (r26-ds-006/component-b), with the notebooks organised by stage from	• On the headset, per-participant LinUCB snapshots are saved as JSON in AudioOnlineModels, decisions and outcomes as JSONL in RLTransitions, and generated MP3s in a cache. The Lyria backend also stores the MP3s and their prompt metadata. • The relay stores session JSONL files, and long-term preferences are designed to be kept in Supabase.	• Datasets and trained checkpoints are excluded from Git, while all code and configuration are versioned in Git. • Checkpoints are versioned through descriptive file names such as fusion_meld_baseline.pt and fusion_acted, and ABLATION_STUDY.md records which data

			01_pipeline to 05_deployment. • The file model_config.json records the source notebook, seed 42, the training subjects, the window count and the LOSO metrics. Every artifact also has a SHA-256 fingerprint, and .gitattributes stops line-ending changes from altering them. DVC is not used.	• Git versions the code, configuration, scene assets and schema IDs (audio-context-v1-40 and audio-micro-actions-v1). Snapshots are checked for owner, configuration, schema and matrix size before loading, and DVC and Git-LFS are not used.	trained each checkpoint. • Session data is stored in an SQLite database (componentd.db) and in JSONL log files, and DVC is not used.
Model Development	**Model Selection**	• The adaptive condition uses a disjoint LinUCB contextual bandit, with one interpretable linear model for each of 11 safe discrete visual actions. • LinUCB was chosen because it personalises online with few samples, explores based on uncertainty, can be audited deterministically and has a low CPU and memory cost on Quest 2. • Static-personalised and rule-based policies are kept as study baselines.	• The final model is a two-way ensemble of XGBoost (weight 0.15) and MS-CGCA (weight 0.85). MS-CGCA is a causal network with dilated causal Conv1D layers (dilations 1, 2 and 4), an	• The active model is a per-participant disjoint LinUCB contextual bandit over 13 audio micro-actions. It can learn from sparse, delayed outcomes, and unsafe actions are masked before selection. • It is compared with two controllers, StaticPersonalized and RuleBasedAdaptive.	• The model combines a frozen emotion2vec_plus_large encoder (1024-dimensional) with a trainable prosody branch, a gated fusion layer and a learned valence/arousal head.

		Deep RL methods were deferred because there is no large participant dataset and online exploration must stay safety-constrained. • The choice is an architectural one, since comparative pilot results are not yet available.	LSTM(128) and attention whose query comes only from time-of-day features. • It was compared with XGBoost alone, a BiLSTM, a CNN-LSTM, CORN ordinal heads, a three-way ensemble with a personalised head and a 120-beat version. • Models were compared with leave-one-subject-out (LOSO) macro-F1 and Wilcoxon tests. MS-CGCA matched the BiLSTM while being causal, the personalised head was dropped because its gain was not significant	Earlier PPO, TD3 and SAC experiments used simulated users and are not deployed. • The choice is architectural, since no controlled comparison with real users has been recorded yet.	• emotion2vec was chosen because it reached 84.7% on acted binary stress detection, compared with 60.9% for wav2vec2. • The learned valence/arousal head replaced the rule-based emotion-to-stress lookup from PP1. The final model was selected on real-voice results, because metrics on acted speech did not hold up on real voices.

			(+0.0075, p = 0.625), and 60-beat windows halved latency compared with 120 beats. The blend weights were chosen inside nested cross-validation.		
	Model Training	• The model is a custom double-precision C# LinUCB implementation that runs on the Quest 2 CPU, without PyTorch, TensorFlow or a GPU. • It uses 11 arms and 24 features, with ridge regularisation 1.0 and exploration coefficient 0.25. The design matrix starts as the identity, the reward vector starts at zero and there is no forgetting. • The reward weights are 1.00 for stress, 0.35 for RMSSD, 0.15 for heart rate, 0.10 as an environment-change penalty and 2.00 for safety interventions. • Because learning is online, there is no train/test split, random seed or epoch count. Each session starts a fresh model with about 11 decisions in the 900-second adaptive phase, and only the executed action is	• Training was done in a Kaggle notebook on a Tesla T4 GPU, using TensorFlow 2.20 (Keras 3) and XGBoost 3.x. • XGBoost uses 300 trees, depth 6, a learning rate of 0.05 and a subsample and column-sample rate of 0.8. • MS-CGCA is trained with Adam (learning rate 1e-4), focal loss ($\gamma = 2$), batch size 32 and class weights for up to 120 epochs,	• The model is trained incrementally on the Unity runtime CPU during participant sessions. • Each action has a 40 × 40 ridge matrix with ridge 1.0 and exploration $\alpha = 0.25$. Each adjustment is a 0.03 micro-step, decisions are at least 75 seconds apart, outcomes settle for 5 seconds and time out after 240 seconds, and only the executed action is updated. • There is no offline split, seed or fixed training time, and no real-user accuracy or reward	• Training uses PyTorch 2.x in two stages: features are cached once on a Colab GPU, and the fusion head is then trained in a few minutes. • The settings are batch size 64, learning rate 1e-3, weight decay 1e-4, dropout 0.3 and up to 100 epochs, with early stopping (patience 10) on validation CCC. The anomaly VAE is trained locally.

		updated after a valid delayed reward. • Accuracy and F1 do not apply until the pilot runs. Automated tests instead check the scoring maths, updates, reward attribution and failure gates.	with early stopping (patience 15) on a 15% validation split and seed 42. • The model was evaluated with 15-fold LOSO, which took about 22 minutes, and then refit on all 15 subjects in 1.9 minutes (33 epochs). • On unseen subjects, the deployed model reaches a nested-LOSO macro-F1 of 0.5923 and a quadratic weighted kappa of 0.7525.	metric is available yet. Results from the historical simulator are not reported as current performance.	• The shipped model reaches 91.7% binary accuracy on real voices (92% English and 91% Sinhala) at a threshold of 2.0 chosen with leave-one-out Youden's J, with $d' = 3.62$. On held-out acted speech it reaches CCC 0.864 for valence, 0.810 for arousal and F1 0.92.
Model Integration	**Tools used**	• LinUCB is compiled into the standalone Unity app and called on the device through the C# IEnvironmentPolicy/PolicyController pipeline, so no REST, ONNX or Sentis runtime is needed. • Physiology from the HRV stress inference component arrives through .NET ClientWebSocket at /stream,	• The model is served by a FastAPI 0.143 and uvicorn 0.54 backend on Python 3.13. It exposes the WebSocket endpoints /ingest and /stream and	• Unity 6000.0.82f1 runs the C# policy, safety filter, mixer, playback and the JSON/JSONL persistence. • Preferences arrive through the WebSocket session relay, and HRV-based stress windows	• The model is served as a REST API with FastAPI and uvicorn on port 8010, with the endpoints /ambient-check, /voice-turn, /full-session, /sessions,

		while mobile commands and outgoing telemetry use the relay at /realtime?role=quest. • The main stack is Unity 6000.0.82f1, URP 17.0.4, OpenXR 1.16.1, XR Interaction Toolkit 3.0.11 and Input System 1.19.0, with JsonUtility for serialisation.	the REST endpoints /stress/latest and /health, with Pydantic 2.14 message schemas. • The models are loaded in-process from a Keras .keras file, an XGBoost booster JSON and a pickled scikit-learn scaler, and NeuroKit2 handles PPG peak detection. • Each output message contains the stress level or band, a confidence value, all four class probabilities and a band flag, which is set when the top two probabilities are within $\tau = 0.15$.	arrive through a dedicated physiology bridge. • A local Python FastAPI/Uvicorn service on port 8002 calls Lyria through google-genai and returns the MP3 as base64 over REST. Unity then crossfades it with the ambient track, and no ONNX or Sentis runtime is used.	/companion/tts and /health. • The React Native 0.87.1 app calls the API through a typed client, using adb reverse during development. • The backend polls the HRV stress inference backend's /stress/latest endpoint to cross-check the voice result against HRV. • The companion uses faster-whisper for speech-to-text, Ollama or Claude as the language model and ElevenLabs or on-device text-to-speech. The PyTorch model runs on the server, and

					ONNX is not used.
Model Deployment	**Testing Environments**	• The project contains 464 EditMode tests in 69 files and 27 PlayMode tests in 5 files, written with Unity Test Framework 1.6.0 and NUnit. They cover feature construction, LinUCB maths, physiology validation, safety, rewards, networking and telemetry. • A full-suite pass was recorded on 30 August 2026. • Development testing uses the Unity Editor with local mock WebSocket integration, and Quest 2 pairing has been validated. Full-duration APK runs, thermal profiling, Wi-Fi-loss recovery and emergency-stop tests on the headset are still pending.	• A pytest suite of 75 tests was re-run on 14 October 2026, with 74 passing and 1 skipped. It covers the API, causality, the engine, feature parity, temperature handling and model parity, where 200 reference windows match with a maximum probability difference of 5.4e-07 and 100% argmax agreement. • A timing test showed that each 15-second frame is processed in under 0.5 seconds on a 2-core CPU. • An end-to-end test sends frames	• The test-audio-online.ps1 script runs 23 deterministic C# tests, and 5 Python aggregation tests also pass (verified on 14 October 2026). • A synthetic sanity check improved preferred-action choices from 17/20 to 20/20, but this only checks the software and does not show effectiveness for participants. • Editor playback, the full wearable-to-mobile-to-Quest integration and real-user comparisons have not yet been verified.	• There are 143 pytest functions across 17 test files, covering unit, layer and API integration tests. • Separate evaluation scripts measure performance on Sinhala, on real voices and for ambient-noise calibration. • Testing is done in a local Python 3.12 virtual environment and on a Samsung Galaxy A9, where a live five-layer session was confirmed on 30 August 2026.

			to /ingest while a second client listens on /stream. The system has also been tested on real hardware with the wearable, an Android phone and the Quest 2 on the same network.		
	Deployment Platform	• The final runtime is the Meta Quest 2 standalone headset (Android, IL2CPP, ARM64, OpenXR), and JapaneseTemplePondGarden.unity is the only production scene. • Policy selection, safety checks, visual transitions, reward attribution and telemetry all run on the headset. The HRV stress inference backend and the session relay run on a laptop on the same LAN.	• The backend runs on a laptop that acts as a local server (Windows for demos and Linux for testing) on the same Wi-Fi network as the phone and the Meta Quest 2. • It runs on Python 3.13 using only the CPU, and the model does not run on the phone or the headset.	• The policy is packaged inside the shared Unity app, which targets the Meta Quest 2 (Android IL2CPP/ARM64), and development is done in the Windows Unity Editor. • The Lyria backend and session relay run as Python services on a local PC, and there is no cloud deployment.	• The backend runs on a laptop server using Python 3.12 and uvicorn on port 8010, and the model does not run on the phone or headset. • The client is the React Native Android app on the Galaxy A9, and no cloud host has been set up yet.
	Deployment Method	• The Unity project is built as an ARM64 IL2CPP Android APK (version 0.1.0, package	• The model is not containerised. It ships as files in	• The Unity Android APK is built from JapaneseTemplePondGa	• The backend is started with uvicorn

		com.mindsyncvr.templepond) and installed on the Quest 2, with the LinUCB code and ScriptableObject profiles bundled inside it. • Running it requires the HRV stress inference backend on port 8000, the session relay on port 8080, the configured LAN host and a six-digit pairing code from the mobile app. • Before production, the development ws:// endpoints must be replaced with authenticated wss:// endpoints. A final release APK has not yet been produced.	artifacts/ (xgb_population.json, mscgca_population.keras, feature_scaler.pkl and model_config.json), and the loader checks the configuration at startup. • To launch it, the dependencies are installed with pip install -r requirements.txt and the server is started with uvicorn server.main:app --host 0.0.0.0 --port 8000 (or the group's demo-backends.cmd). Clients then connect to ws://<laptop-ip>:8000.	rden.unity, where the scene installer connects AudioRLAgent, the preference receiver, the ambient track and the clip start gate. • The Lyria backend (port 8002), session relay (port 8080) and the HRV stress inference backend (port 8000) are started separately, and GEMINI_API_KEY and the host address are kept in untracked environment settings. • The mobile app must send the study condition and the participant's preferences before the personalised model or the first music clip is prepared.	server.main:app --host 0.0.0.0 --port 8010 inside a virtual environment built from requirements.txt. • The checkpoints fusion_meld_baseline.pt (about 710 KB) and anomaly_v2.pt (about 10 KB) are bundled with the backend, and the emotion2vec weights download automatically on first run. • API keys stay on the server as environment variables. The app is installed as an Android APK, and adb reverse tcp:8010 connects it to the backend; Docker is not used.

Future Enhancements	**Model Improvement:**	• The next step is to run the two-day counterbalanced pilot and use the participant outcomes to calibrate exploration, regularisation, reward weights, quality gates and visual step sizes. • Larger and more diverse samples are needed, and cross-session personalisation could be added once privacy approval is in place. • A Forest Lake scene can be added after the single-scene version is stable. • Full-duration performance and safety validation on Quest 2 also needs to be completed.	• More diverse participants with self-reported stress labels should be collected to replace the protocol-based labels. • The confidence values should be calibrated, and a real skin-temperature sensor should be added to the wearable. • The server should be secured with TLS (wss://), authentication and restricted CORS. • The current prediction delay of about 45 seconds should be reduced.	• Consented, counterbalanced real sessions should be run in all three controller conditions, keeping visuals fixed so the audio effect is isolated. Reward, stress and HRV change and intervention rate should be reported for each participant. • The reward weights, α, thresholds and action sizes should be calibrated on real outcomes. • Cold-start behaviour, network loss, failed clip generation and persistence on Quest should be tested, and data-retention controls and TLS authentication should be added.	• The model should be retrained with real continuous valence/arousal labels from IEMOCAP or MSP-Podcast, because the frozen encoder struggles with arousal on quiet stress and unfamiliar speakers. • The ambient-noise thresholds should be calibrated on real Galaxy A9 recordings, and the severity bands should be validated. • An out-of-distribution confidence term should be added, Tamil should be evaluated and server-side text-to-speech should be added.